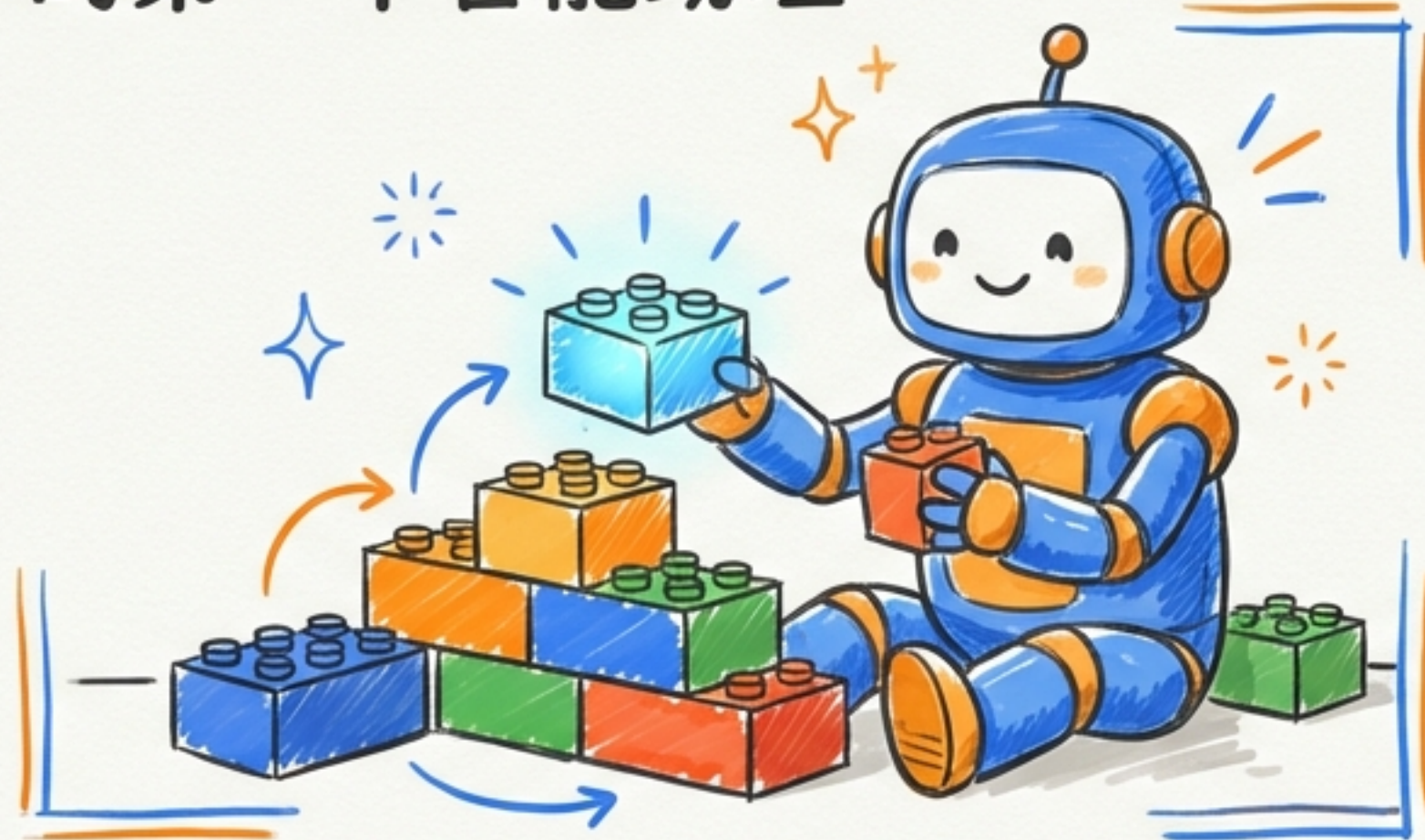
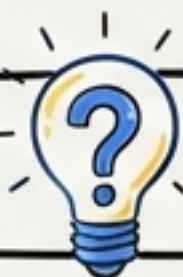


基于LangGraph搭建一个Agent应用

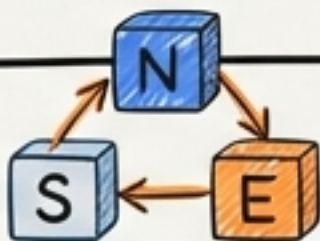
像搭乐高一样，构建你的第一个智能助理



我们的建造蓝图



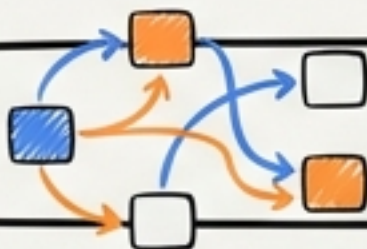
1. 为什么选择图结构？：从简单的“链式”思维，升级到复杂的“网状”决策。



2. 核心概念：掌握LangGraph的三大基石：状态 (State)、节点 (Node) 和边 (Edge)。



3. 备料与施工：准备环境，定义工具、模型和Agent核心逻辑。



4. 组装与连接：一步步将节点和边搭建成一个完整的Agent工作流图。



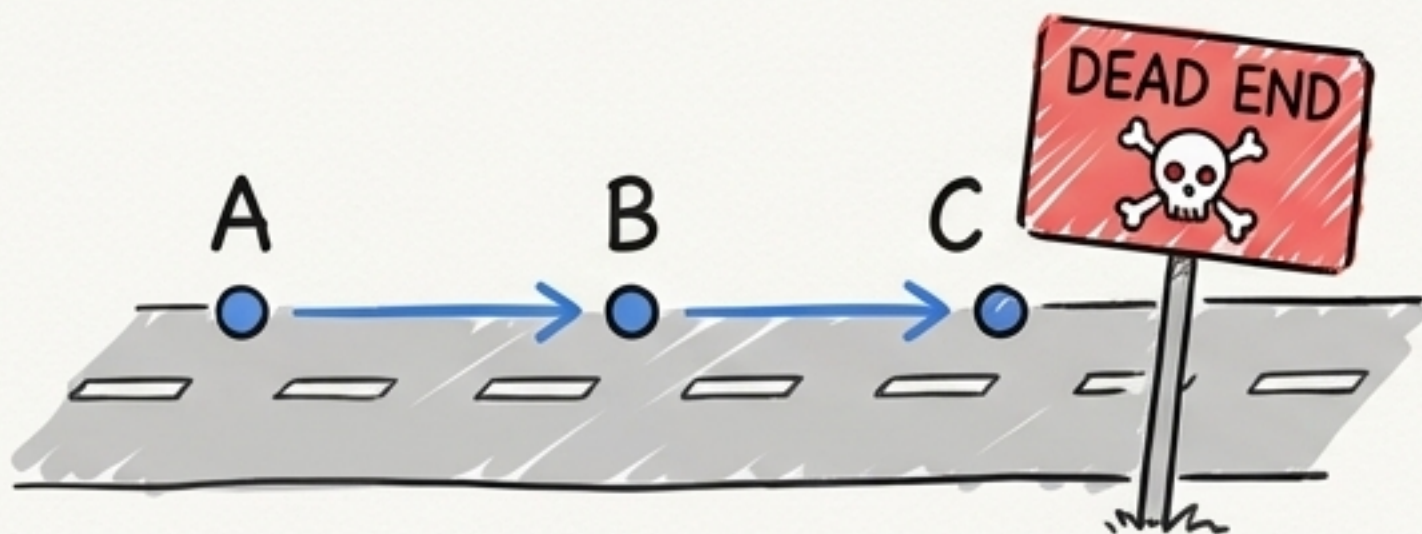
5. 运行与验证：启动我们的Agent，观察它的工作流程，并将其可视化。



6. 总结与展望：回顾核心模式，探索更多可能。

为什么需要“图”？从链式到网状的思维跃迁

传统链式（如LCEL）的局限



像一条单行道，只能 $A \rightarrow B \rightarrow C$ 顺序执行。无法轻松实现循环、条件判断和更复杂的逻辑，这对于需要思考和使用工具的Agent来说是致命的。

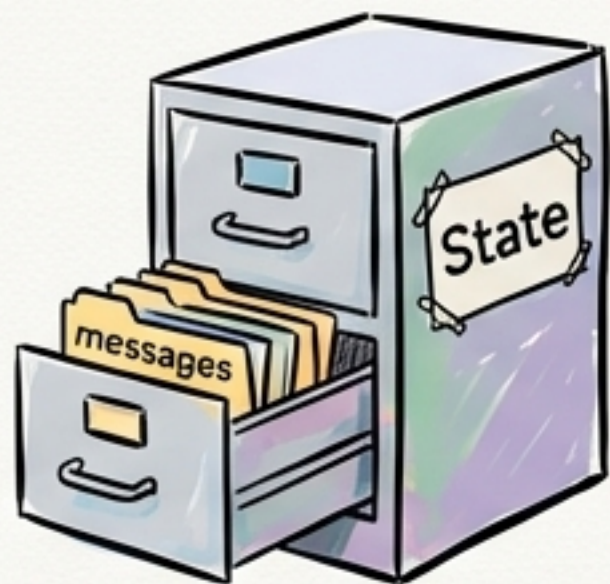
Agent的真实需求



Agent的工作方式更像是在一个复杂的园区里游览。它需要根据当前情况（“我想看海豚”）决定下一步去哪里，可能会走回头路，或者在多个地点之间选择。这种动态流程需要一个“图”结构来管理。

LangGraph就是为构建这种复杂的、有状态的、可循环的AI工作流而生的。

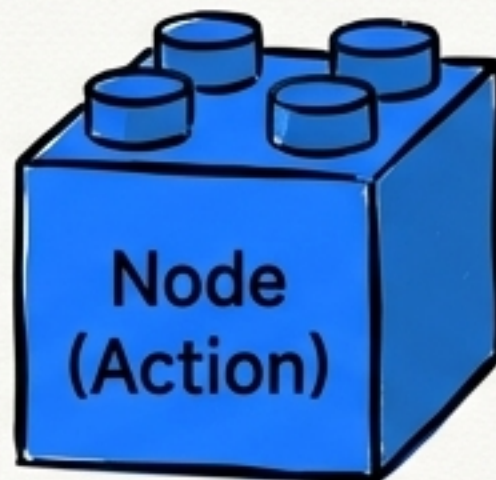
LangGraph的三大核心基石



1. 状态 (State)

- 比喻：Agent的“记忆中心”或“全局记事本”。

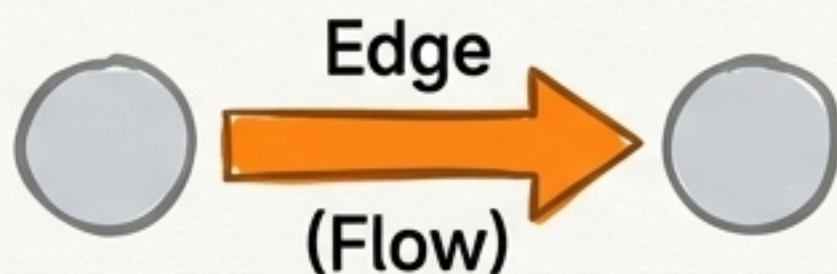
一个贯穿始终的对象，用于在图的各个节点之间传递信息。我们这里使用 `MessagesState`，它会累积所有的对话消息，确保Agent不会“失忆”。



2. 节点 (Nodes)

- 比喻：“功能房间”或“工作站”。

代表图中的一个计算步骤或一个动作。每个节点都是一个接收State作为输入并返回更新后State的函数。例如：“调用大模型进行思考”或“执行一个工具”。



3. 边 (Edges)

- 比喻：连接房间的“走廊”和“楼梯”。

定义了节点之间的连接关系，决定了工作流的走向。可以是固定的（A节点完成后总是去B节点），也可以是带条件的（根据A节点的结果，决定去B节点还是C节点）。

Step 1: 准备“建筑材料”（环境配置）

我们需要安装几个关键的Python库来开始我们的项目。

核心框架

`pip install langgraph` # <-- 注释: 我们的核心骨架 🏗️

与OpenAI兼容的模型接口

`pip install langchain-openai` # <-- 注释: 与“设计师”沟通的电话 📞

我们将要使用的工具依赖

`pip install langchain_core` # <-- 注释: 基础工具箱 🧰

用于持久化记忆

`pip install langchain` # <-- 注释: 持久化支持 💾

Step 2: 聘请“总设计师”并打造“工具箱”



定义工具 (The Toolbox)

Agent的强大之处在于它能使用工具。我们定义一个简单的搜索工具，用于查询天气。

`@tool` 装饰器能让气个 `@tool` 装饰器能让 LangChain 自动识别它。

Python Code

```
from langchain_core.tools import tool

@tool ← 关键! 自动识别
def search(query: str):
    """一个简单的搜索工具"""
    if "上海" in query.lower():
        return "上海现在25度，今天可能有雨。"
    return "现在是35度，阳光明媚。"

tools = [search]
```



初始化并赋能模型 (The Architect)

我们选择一个大语言模型作为Agent的“大脑”。这里我们使用兼容OpenAI接口的DeepSeek模型。关键一步是使用 `.bind_tools(tools)`，告诉模型 `.bind_tools(tools)`，告诉模型它拥有哪些工具。

Python Code

```
from langchain_openai import ChatOpenAI

# 从智谱AI等平台获取API信息
base_url = "https://api.siliconflow.cn/v1/"
api_key = "sk-..."

model = ChatOpenAI(
    base_url=base_url,
    api_key=api_key,
    model="deepseek-ai/deepseek-v2"
).bind_tools(tools)
```

关键步骤：
赋予工具
使用能力!

Step 3: 设计核心功能节点

中央控制室 (Agent节点)

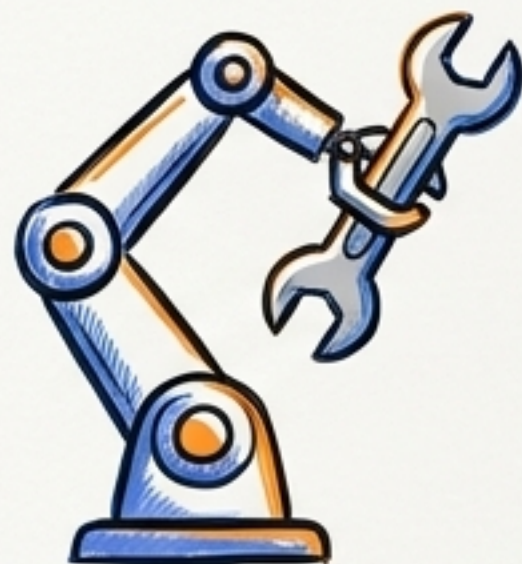


这是Agent的“思考”节点。它接收当前的对话历史，调用大模型，然后决定下一步是直接回答还是使用工具。

```
from langgraph.graph import MessagesState

def call_model(state: MessagesState):
    messages = state['messages']
    response = model.invoke(messages)
    # 将模型的回复也加入到状态中
    return {"messages": [response]}
```

执行部门 (Tool节点)



LangGraph提供了一个预构建的`ToolNode`，它是一个非常方便“执行”节点。当`call\_model`节点决定使用工具时，`ToolNode`会自动解析并执行相应的工具函数（比如我们的`search`函数），然后将结果返回。

```
from langgraph.prebuilt import ToolNode

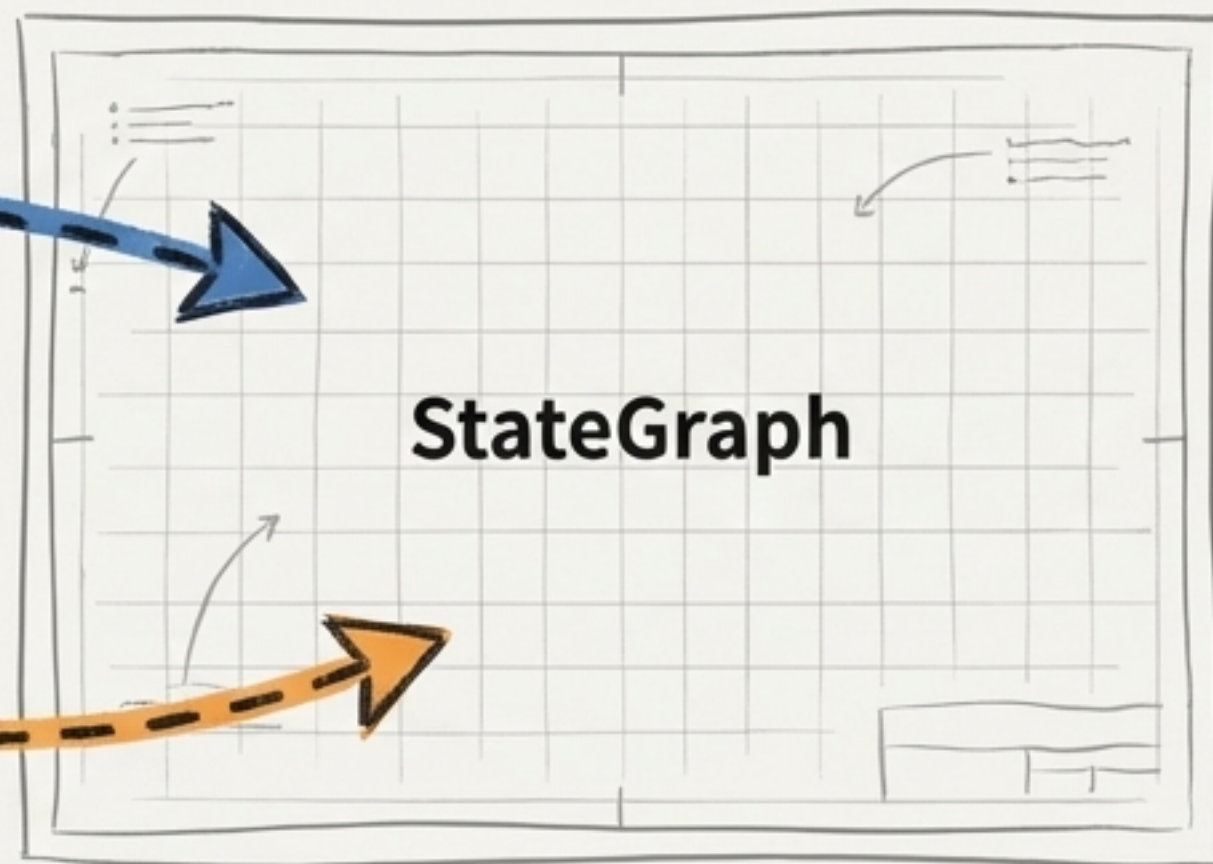
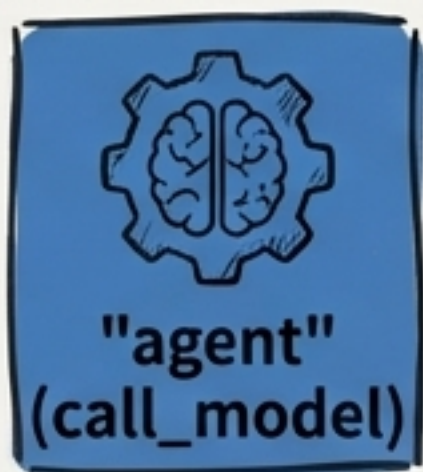
tool_node = ToolNode(tools)
```

Step 4: 绘制施工图并放置功能模块

第一步：创建蓝图

我们首先实例化一个`StateGraph`。它需要知道我们的“全局记事本”是什么类型，这里是`MessagesState`。现在我们有了一个空的施工图。

```
workflow = StateGraph(MessagesState)
```



第二步：放置节点

使用`.add\_node()`方法，将我们之前定义的“中央控制室”(agent)和“执行部门”(tools)两个功能模块放置到施工图上。每个节点都有一个唯一的名字。

```
# 添加agent节点，它会执行call_model函数
workflow.add_node("agent", call_model)
# 添加tools节点，它会执行tool_node的功能
workflow.add_node("tools", tool_node)
```

Step 5: 连接节点，定义 workflows



定义路由逻辑 (The Brain of the Flow)

我们需要一个函数来判断“agent”节点完成后，下一步该去哪里。逻辑是：检查最后一条消息是否包含工具调用请求。如果有，就去“tools”节点；如果没有，就结束（END）。

```
def should_continue(state: MessagesState) -> str:
    last_message = state['messages'][-1]
    # v--
    if last_message.tool_calls:
        return "tools"
    # v--
    return "end"
```

去执行工具

结束流程



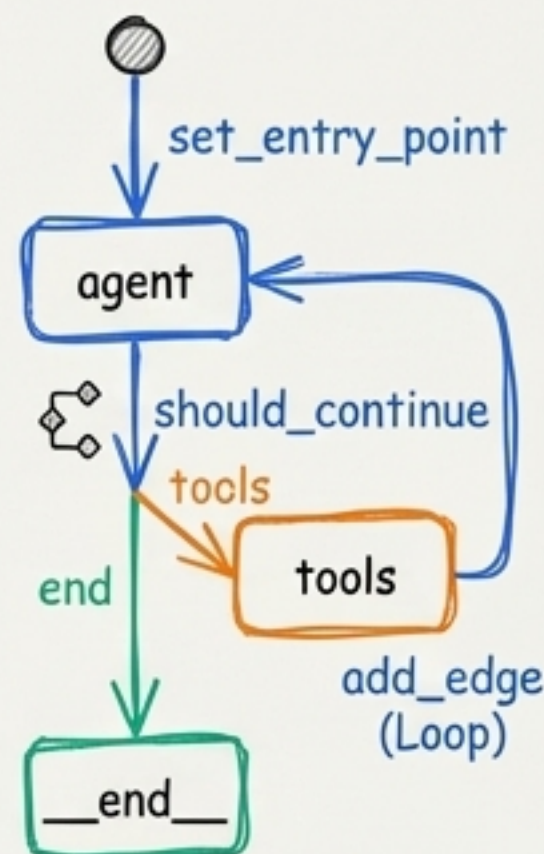
连接蓝图 (Wiring the Blueprint)

我们用`set\_entry\_point`定义从哪里开始，用`add\_conditional\_edges`添加我们的路由逻辑，最后用`add\_edge`创建一个从“tools”返回“agent”的闭环。

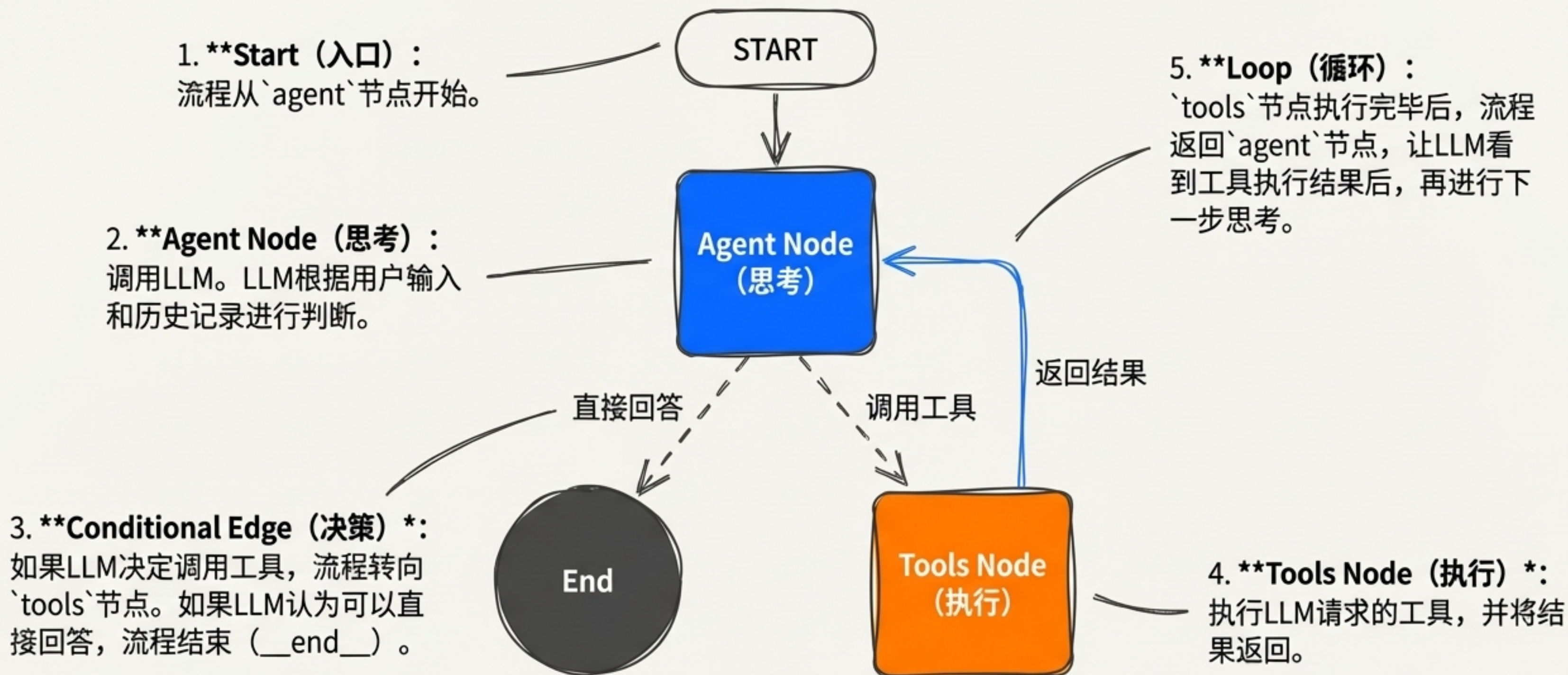
```
# 设定入口
workflow.set_entry_point("agent")

# 添加条件边
workflow.add_conditional_edges(
    "agent",
    should_continue,
    # 映射函数返回值到节点名
    {"tools": "tools", "end": "__end__"}
)

# 添加常规边，形成闭环
workflow.add_edge("tools", "agent")
```



我们的Agent架构，组装完毕！

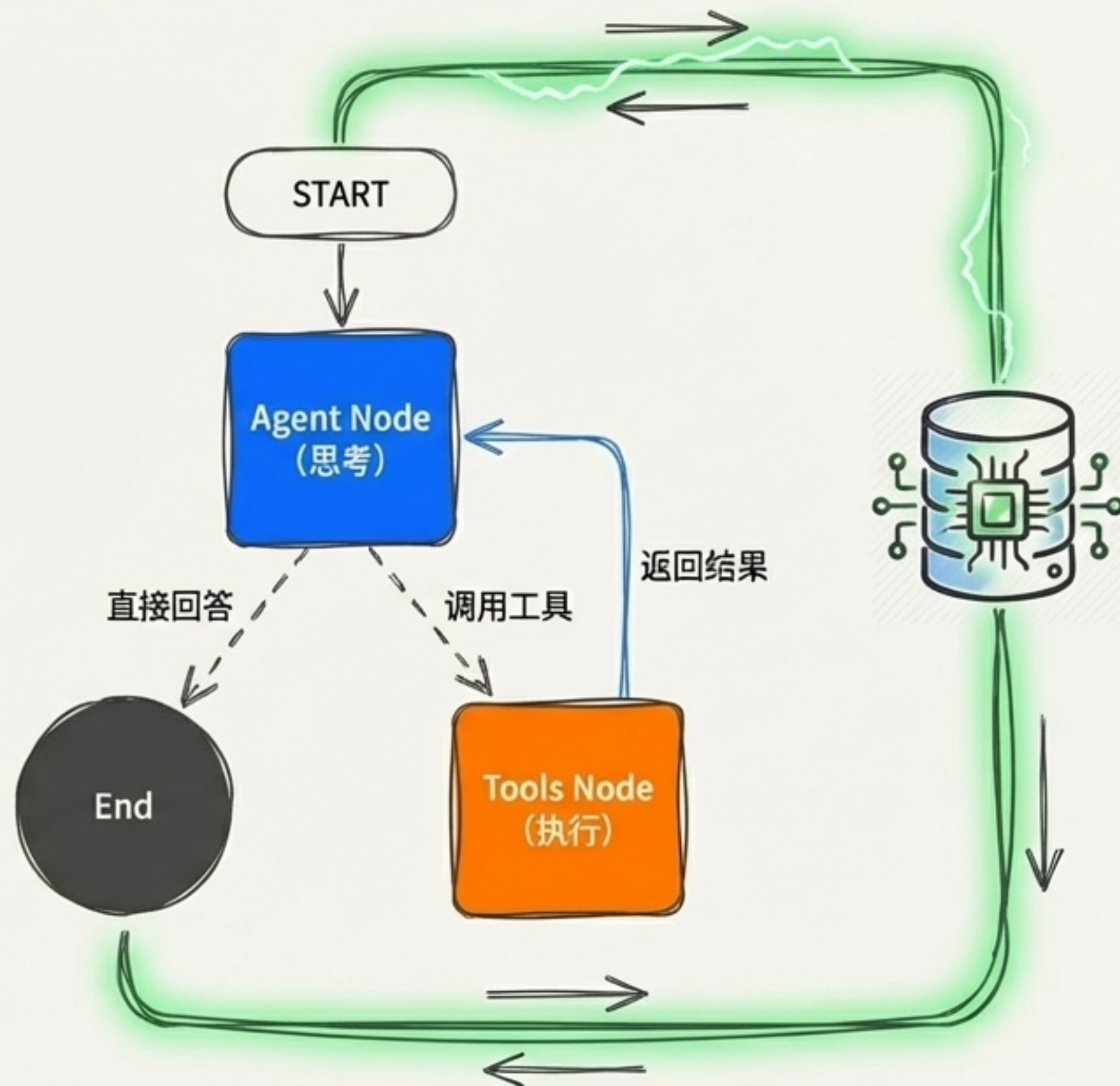


Step 6: 安装“记忆芯片”（启用持久化）

到目前为止，我们的Agent每次运行都是一次全新的对话。为了让它能记住之前的交流（实现多轮对话），我们需要添加一个Checkpoint。`MemorySaver`是一个简单的内存检查点，它会将每个线程（thread）的对话状态保存在内存中。

```
from langgraph.checkpoint.memory import MemorySaver  
checkpointer = MemorySaver()
```

关键点：在编译图时，将`checkpointer`传入。这样LangGraph就会自动为我们处理状态的保存和加载。



Step 7: 竣工! 编译并运行Agent

编译 (Compile)

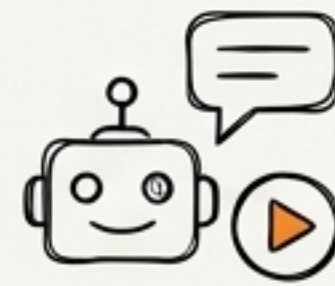
`.compile()`方法会将我们定义的所有节点和边最终“编译”成一个可执行的应用。我们将之前创建的`checkpointer`在这里传入。



```
app = workflow.compile(checkpointer=checkpointer)
```

运行 (Invoke)

我们使用`.invoke()`来和Agent交互。输入是一个包含`messages`的字典。`configurable`中的`thread\_id`是关键，它让Checkpointer知道这是哪一个对话线程，从而加载正确的历史记录。



```
# 提一个需要使用工具的问题
input_messages = {
    "messages": [("user", "上海的天气怎么样?")]
}
config = {"configurable": {"thread_id": "user_1"}}

# final_state 将包含完整的对话历史
final_state = app.invoke(input_messages, config)

# 打印最后一条消息
print(final_state['messages'][-1].content)
```

内部探秘：追踪Agent的“思考”路径

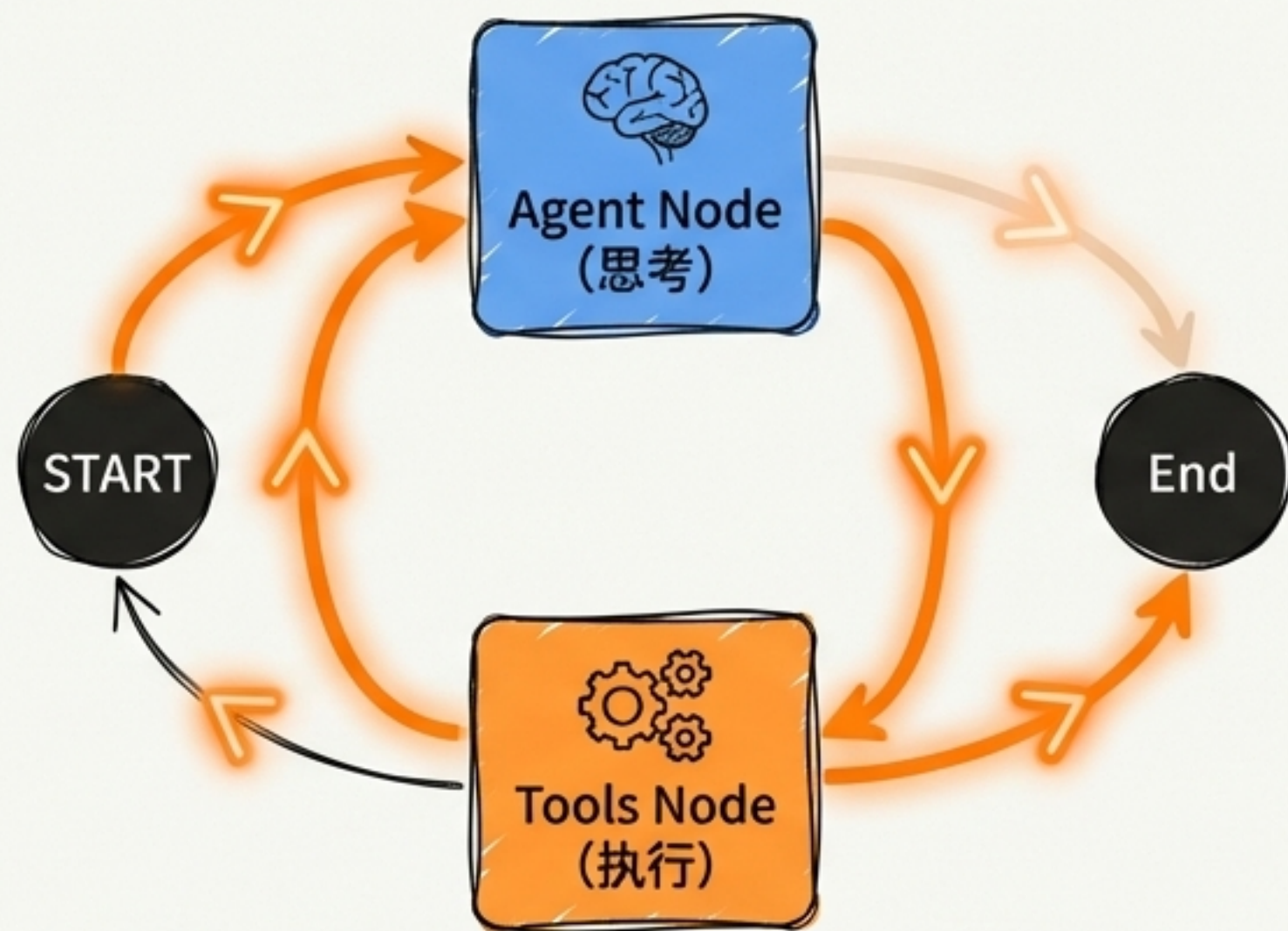
案例：用户提问“上海的天气怎么样？”

执行日志（模拟）

```
> Invoking Agent...  
[Agent Node] LLM decided to use 'search'  
tool with query '上海天气'.  
> Path leads to 'tools'.  
[Tools Node] Executing search(query='上海天  
气'). Result: '上海现在25度, 今天可能有雨.'  
> Path leads back to 'agent'.  
[Agent Node] LLM sees tool result and  
generates final answer.  
> Path leads to '__end__'.
```

Final Answer: 根据我查询到的信息, 上海目前气温为25度, 并且今天可能会下雨。

路径可视化



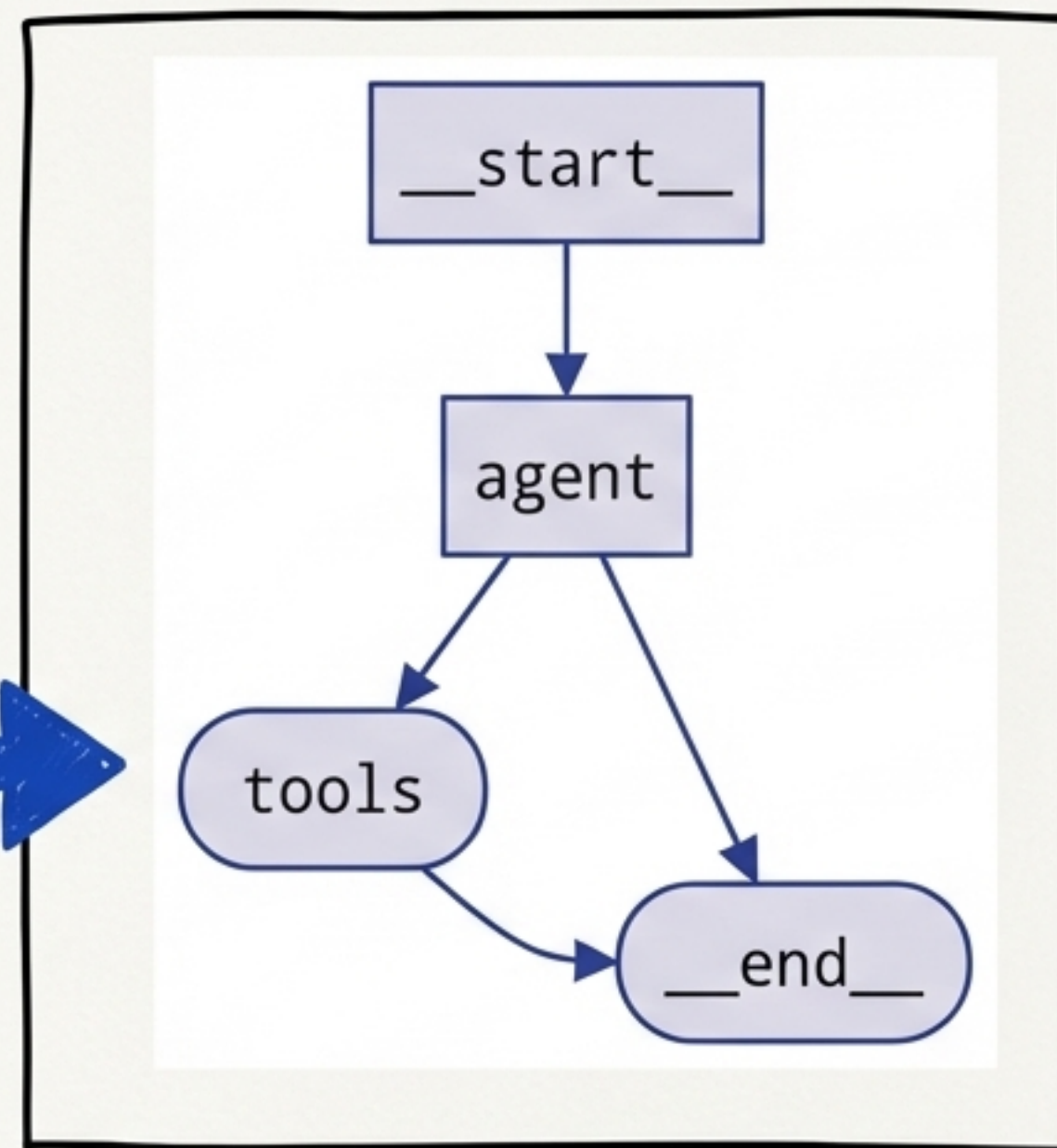
生成可视化报告：让Graph自己画出自己

代码实操讲解

LangGraph内置了强大的可视化功能。只需几行代码，就可以将我们构建的图生成一张图片，这对于理解、调试和文档化复杂的Agent非常有帮助。

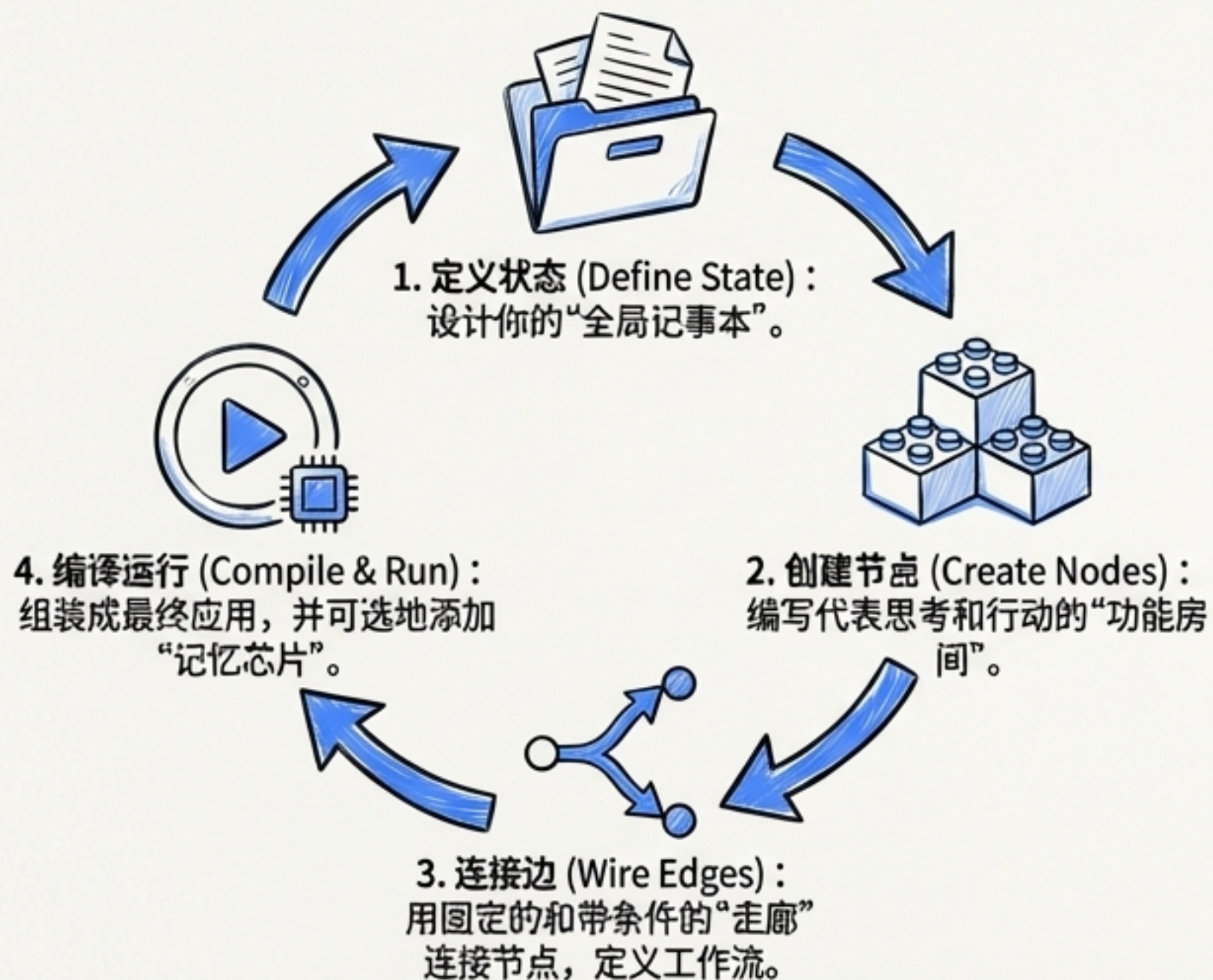
```
# 需要安装 `pygraphviz` 或 `pydot`  
# pip install pygraphviz  
from IPython.display import Image  
  
# 生成PNG图片  
png_data = app.get_graph().draw_png()  
with open("agent_graph.png", "wb") as f:  
    f.write(png_data)  
  
Image(png_data)
```

生成的图



核心模式回顾 & 你的下一个Agent

LangGraph Agent的核心建造模式



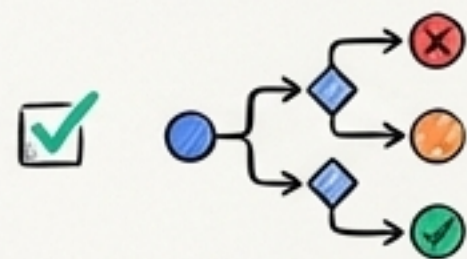
挑战与展望：构建你自己的Agent



增加更多工具：例如，一个计算器工具，一个数据库查询工具。



引入“人工审核”节点：在某些危险操作 (如删除文件) 前，增加一个需要人类确认的节点。



实现更复杂的路由：例如，根据错误类型，决定是重试还是向用户求助。



集成更强大的模型：尝试不同的LLM，看看它们如何影响Agent的行为。